



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Sección 1

Análisis Léxico

Equipo: **VISIONLY**

Mantenlo simple!

Profesor:
Félix Márquez

Integrantes:
Ana Santamaria V-21251440
Carlos Martínez V-27955202
Genesis Moya V-31370339
Naleska Carrera V-31521506

Ciudad Guayana, julio 15 del 2026.

INTRODUCCIÓN

El análisis léxico constituye la primera fase fundamental en el proceso de compilación, encargada de transformar una secuencia de caracteres en una secuencia de tokens que serán utilizados por las etapas posteriores del compilador. En el marco de la asignatura Lenguaje y Compiladores, el presente informe documenta el desarrollo de cuatro actividades prácticas que nos permitieron explorar tanto los fundamentos teóricos como las aplicaciones concretas del análisis léxico.

En la primera actividad, abordamos los Autómatas de Pila (PDA) como máquinas teóricas capaces de reconocer lenguajes libres de contexto, comprendiendo su superioridad frente a los autómatas finitos deterministas (AFD) y no deterministas (AFND)

La segunda actividad nos llevó a construir un analizador léxico manual para la verificación de archivos de configuración de Docker, utilizando expresiones regulares en Python.

En la tercera actividad, definimos un subconjunto del lenguaje Rust denominado MiniRust y construimos su analizador léxico utilizando el metacompilador Logos, una herramienta moderna del ecosistema Rust que genera autómatas finitos deterministas a partir de la definición de tokens mediante atributos y expresiones regulares.

Finalmente, en la cuarta actividad, investigamos las aplicaciones de Flex en el área de seguridad informática, identificando su uso en herramientas como sistemas de detección de intrusos (Snort), análisis de logs y firewalls, donde el análisis léxico permite identificar patrones maliciosos en el tráfico de red y en archivos de registro.

El presente informe recoge, por tanto, el resultado de nuestro esfuerzo por integrar la teoría de autómatas con la práctica del desarrollo de software, evidenciando cómo los conceptos abstractos adquieren relevancia en la construcción de herramientas reales. Cada actividad representó un desafío que nos exigió coordinar esfuerzos, depurar código y aprender de los errores, consolidando así nuestro aprendizaje en el área de los compiladores.

1) AUTÓNOMA DE PILAS.

1.1 DEFINICIÓN FORMAL DE UN AUTÓMATA DE PILA (AP)

Un Autómata de Pila es una máquina teórica de estados que extiende las capacidades de los autómatas finitos mediante la incorporación de una memoria auxiliar en forma de pila. Esta memoria funciona bajo el principio LIFO (*Last-In-First-Out*), lo que significa que el último símbolo en entrar es el primero en salir. A diferencia de los autómatas finitos, cuyas transiciones dependen únicamente del estado actual y el símbolo de entrada, las transiciones de un AP dependen también del símbolo en el tope de la pila.

Matemáticamente, un Autómata de Pila Determinístico (APD) se define rigurosamente mediante la siguiente 7-tupla: $M = (E, A, P, \delta, e_0, Z_0, F)$

Donde cada componente representa: $\rightarrow$

- E : Un conjunto finito de estados internos de la máquina.
- A : El alfabeto de entrada, conformado por el conjunto finito de símbolos que el autómata puede leer de la cinta.
- P : El alfabeto de la pila, que contiene los símbolos que pueden ser almacenados en la memoria auxiliar $P \cap A = \emptyset$.
- δ : La función de transición, definida como $\delta: E \times (A \cup \{\epsilon\}) \times P \rightarrow E \times P^*$. Esta función determina el siguiente estado y la cadena de símbolos que reemplazará al tope de la pila basándose en el estado actual, el símbolo leído (o la cadena vacía ϵ) y el símbolo actual en el tope.
- e_0 : El estado inicial, donde $e_0 \in E$.
- Z_0 : El símbolo inicial de la pila (o símbolo distinguido), donde $Z_0 \in P$.
- F : El conjunto de estados finales o de aceptación, donde $F \subseteq E$.

1.2 EJEMPLO PRÁCTICO DETALLADO: LENGUAJE $L = \{0^n 1^n \mid n \geq 1\}$

Este lenguaje no es regular, ya que requiere "recordar" cuántos ceros se han leído para compararlos con la cantidad de unos, tarea imposible para un autómata finito convencional.

Definición de la Tupla para este caso: $M = (\{q_0, q_1\}, \{0, 1\}, \{x, \#\}, \delta, q_0, \#, \emptyset)$

En este diseño, la aceptación se produce por pila vacía al procesar un símbolo especial de fin de cadena (usualmente representado como \$) o simplemente al agotar la pila tras leer la entrada.

Lógica de funcionamiento:

- Empuje (Push):** Al leer cada '0', se introduce un símbolo 'X' en la pila para contabilizar la aparición del carácter.
- Extracción (Pop):** Al comenzar a leer los '1', por cada uno de ellos se extrae una 'X' de la pila.
- Validación:** Si al finalizar la lectura de la entrada la pila solo contiene el símbolo inicial (o queda vacía tras extraerlo), la cadena es aceptada.

Tabla de Transiciones:

Estado Actual	Símbolo Entrada	Tope de Pila	Siguiente Estado	Acción en la Pila (Reemplazo)	Lógica
q_0	0	#	q_0	$X\#$	Primer '0': apila X y mantiene fondo
q_0	0	X	q_0	XX	Subsiguientes '0': apila otra X
q_0	1	X	q_1	ϵ	Primer '1': cambia de estado y desapila X
q_1	1	X	q_1	ϵ	Subsiguientes '1': sigue desapilando X
q_1	\$ (Fin)	#	q_1	ϵ	Si fin y queda fondo, vacía pila (Acepta)

1.3 PODER COMPUTACIONAL E IMPORTANCIA (RELACIÓN CON AFD Y AFND)

En la Jerarquía de Chomsky, los Autómatas Finitos (AFD y AFND) están limitados a reconocer Lenguajes Regulares (Tipo 3), los cuales no poseen memoria de capacidad ilimitada y solo pueden modelar patrones basados en estados finitos. El Autómata de Pila representa un salto cualitativo al ser el reconocedor de los Lenguajes Libres de Contexto (Tipo 2).

Diferencias fundamentales de poder:

- **Memoria:** Mientras que un AFD solo "recuerda" mediante el cambio de estado, el AP utiliza la pila para gestionar estructuras de profundidad arbitraria.
- **Quiebre de la Equivalencia Determinística:** En los autómatas finitos, un AFND puede convertirse siempre en un AFD equivalente. Sin embargo, en los autómatas de pila, los no determinísticos (APND) son estrictamente más poderosos que los determinísticos (APD). Existen lenguajes, como los palíndromos de longitud ambigua, que solo pueden ser reconocidos por un APND.
- **Práctica en Compiladores:** A pesar del mayor poder de los APND, en la construcción de compiladores reales se utilizan variantes de los APD (como los analizadores LR o LL) debido a su eficiencia computacional y predictibilidad, evitando el retroceso (*backtracking*) innecesario.

1.4 CONCLUSIÓN SOBRE LAS UTILIDADES DEL AUTÓMATA DE PILA

El impacto de los autómatas de pila en la ingeniería de software es fundamental, especialmente en la fase de Análisis Sintáctico (Parsing) de un compilador. Mientras que el análisis léxico se ocupa de reconocer tokens mediante autómatas finitos (lenguajes regulares), el análisis sintáctico requiere validar la estructura gramatical del código, la cual suele ser libre de contexto.

Se basa en sus utilidades principales de validación de estructuras anidadas que son la herramienta natural para verificar el equilibrio de paréntesis, llaves y corchetes en lenguajes de programación, así como la correcta apertura y cierre de

etiquetas en lenguajes de marcado como HTML y XML. Su evaluación de expresiones se utilizan para la conversión de expresiones infijas a postfijas (notación polaca inversa) y su posterior evaluación aritmética mediante el uso de la pila. Y el procesamiento de lenguajes recursivos le da la capacidad de memoria LIFO permitiendo gestionar las llamadas a funciones y retornos en la ejecución de programas, manteniendo el rastro de los estados previos.

Por tanto, el autómatas de pila es el pilar que permite a las máquinas procesar lenguajes con gramáticas complejas, permitiendo que la programación moderna sea estructurada, jerárquica y legible.

2) LEXER PARA LA VERIFICACIÓN DE ARCHIVOS DOCKER MEDIANTE EXPRESIONES REGULARES.

2.1 HERRAMIENTAS Y TECNOLOGÍA UTILIZADA

Para el desarrollo de este analizador léxico (lexer), se emplearon las siguientes tecnologías:

- **Lenguaje de Programación:** Python 3.13.
- **Entorno de Desarrollo:** Aplicación móvil Pydroid 3 (Android). Se implementó un algoritmo que genera los archivos de prueba en la memoria local para facilitar su lectura en dispositivos móviles.
- **Librería Principal:** Se utilizó el módulo `re` nativo de Python, el cual proporciona soporte para expresiones regulares, pieza fundamental para el proceso de tokenización.

2.2 CONSTRUCCIÓN DEL LEXER PASO A PASO

Paso 1: Definición de los componentes léxicos (Tokens) y Expresiones Regulares: El primer paso consistió en analizar la estructura léxica de un archivo Dockerfile. Se identificaron los patrones de los componentes léxicos y se tradujeron a expresiones regulares (regex) agrupadas en una lista de tuplas:

- **INSTRUCTION:** Palabras reservadas al inicio de la línea (FROM, RUN, CMD, ENV, etc.). Expresión:
`r'^(FROM|RUN|CMD|LABEL|EXPOSE|ENV|ADD|COPY|ENTRYPOINT|VOLUME|USER|WORKDIR|ARG)\b'`
- **COMMENT:** Líneas ignoradas que inician con el símbolo #. Expresión: `r'#.*'`
- **STRING:** Cadenas de texto entre comillas simples o dobles.
- **ARGUMENT:** Valores que acompañan a las instrucciones (rutas, IPs, variables).
- **MISMATCH:** Cualquier carácter que no encaje en las reglas anteriores, representado por un punto `.` para el control de errores léxicos.

Paso 2: Implementación del Autómata con `re.finditer`: Se unieron todas las expresiones regulares en una sola cadena estructurada mediante grupos con nombres (`?P<name>`). Posteriormente, se utilizó la función `re.finditer` junto con la bandera `re.MULTILINE` para iterar sobre el código fuente carácter a carácter y validar las estructuras léxicas del lenguaje.

Paso 3: Estrategia de Control de Errores: Se implementó un autómata capaz de detenerse ante el primer error de lectura. Cuando el analizador detecta un lexema que cae en la categoría de token `MISMATCH`, levanta una excepción `RuntimeError` indicando el carácter no reconocido y el número de línea, simulando un estado fallido del autómata.

2.3 EJEMPLOS DE EJECUCIÓN

A continuación, se presentan 3 casos de prueba evaluados por el analizador léxico:

- **Ejemplo 1: Archivo Docker básico (Válido)**

```
→ Entrada (Docker_ejemplo1.txt):
# Archivo Docker básico
FROM ubuntu:20.04
WORKDIR /app
COPY . /app
```

```
RUN make /app
CMD "python"
```

→ Salida Analizada:

```
=== Analizador Lexico para Docker ===
(Archivos de prueba generados exitosamente en la memoria)

Escribe el nombre del archivo (ej. Docker_ejemplo1.txt) y presiona Enter: Docker_ejemplo1.txt

--- Analizando Docker_ejemplo1.txt ---
('INSTRUCTION', 'FROM', 2, 0)
('ARGUMENT', 'ubuntu:20.04', 2, 5)
('INSTRUCTION', 'WORKDIR', 3, 0)
('ARGUMENT', '/app', 3, 8)
('INSTRUCTION', 'COPY', 4, 0)
('ARGUMENT', '.', 4, 5)
('ARGUMENT', '/app', 4, 7)
('INSTRUCTION', 'RUN', 5, 0)
('ARGUMENT', 'make', 5, 4)
('ARGUMENT', '/app', 5, 9)
('INSTRUCTION', 'CMD', 6, 0)
('STRING', '"python"', 6, 4)
-----

[Program finished]
```

- Ejemplo 2: Archivo Docker con Puertos y Variables (Válido)

→ Entrada (Docker_ejemplo2.txt):

```
FROM python:3.9-slim
ENV PORT=8080
EXPOSE 8080
```

→ Salida Analizada:

```
=== Analizador Lexico para Docker ===
(Archivos de prueba generados exitosamente en la memoria)

Escribe el nombre del archivo (ej. Docker_ejemplo1.txt) y presiona Enter: Docker_ejemplo2.txt

--- Analizando Docker_ejemplo2.txt ---
('INSTRUCTION', 'FROM', 1, 0)
('ARGUMENT', 'python:3.9-slim', 1, 5)
('INSTRUCTION', 'ENV', 2, 0)
('ARGUMENT', 'PORT=8080', 2, 4)
('INSTRUCTION', 'EXPOSE', 3, 0)
('ARGUMENT', '8080', 3, 7)
-----

[Program finished]
```


- **Ejemplo 3: Archivo Docker con Error Léxico (Inválido)**

Se incluyó intencionalmente el símbolo ? al inicio de una instrucción para validar el comportamiento de los estados fallidos.

→ *Entrada (Docker_ejemplo3.txt):*

```
FROM node:14
?RUN npm install
```

→ **Salida Analizada (Detección de Error):**

```
=== Analizador Lexico para Docker ===
(Archivos de prueba generados exitosamente en la memoria)

Escribe el nombre del archivo (ej. Docker_ejemplo1.txt) y presiona Enter: Docker_ejemplo3.txt

--- Analizando Docker_ejemplo3.txt ---
('INSTRUCTION', 'FROM', 1, 0)
('ARGUMENT', 'node:14', 1, 5)
ERROR LÉXICO: '?' inesperado en la línea 2
-----

[Program finished]
```

3) DEFINIR UN LENGUAJE L SUBCONJUNTO DE LENGUAJE RUST, CONSTRUYA UN LEXER PARA L UTILIZANDO METACOMPILADOR.

El análisis léxico es la primera fase en el proceso de compilación, encargada de transformar una secuencia de caracteres en una secuencia de tokens. Para lenguajes complejos como Rust, construir un analizador léxico manualmente puede ser tedioso y propenso a errores. Los metacompiladores o generadores de analizadores léxicos permiten describir los patrones de los tokens mediante expresiones regulares y generan automáticamente el código del lexer.

En esta actividad se ha seleccionado el lenguaje Rust como base, definiendo un subconjunto denominado **MiniRust**, y se ha utilizado el metacompilador **Logos** para construir su analizador léxico.

¿Por qué Logos?

→ Está escrito en Rust y genera código Rust, lo que facilita la integración.

- Es rápido y fácil de usar.
- Permite definir tokens mediante atributos `#[token]` y `#[regex]`.
- Soporte para omisión de espacios, comentarios y manejo de errores.

3.1 MANUAL DE USUARIO DEL METACOMPILADOR LOGOS

3.1.1 ¿Qué es Logos?

Logos es un generador de analizadores léxicos para Rust que permite crear lexers rápidos y eficientes mediante la definición de un enum que representa los tokens del lenguaje. Se basa en macros de Rust y genera un autómata finito determinístico (AFD) a partir de las expresiones regulares definidas.

3.1.2 Instalación

Para usar Logos en un proyecto Rust, se debe agregar la dependencia en el archivo *Cargo.toml*:

```
[dependencies]
```

```
logos = "0.14"
```

3.1.3 Estructura Básica de un Lexer con Logos

1. Definir el enum de tokens con la derivación Logos:

```
use logos::Logos;

#[derive(Logos, Debug, PartialEq)]
pub enum Token {
    // Tokens específicos
    #[token("fn")]
    Function,
    #[token("let")]
    Let,
    // Tokens con expresiones regulares
    #[regex(r"[a-zA-Z_][a-zA-Z0-9_]*")]
    Identifier,
```

```
#[regex(r"[0-9]+")]
Integer,
}
```

2. Configurar la omisión de caracteres (espacios, tabuladores, comentarios): `#[logos(skip r"[ \t\n\f]+")]`

3. Usar el lexer:

```
let input = "fn main() { let x = 42; }";
for token in Token::lexer(input) {
    println!("{}", token);
}
```

3.1.4 Características Avanzadas

- **Tokens con datos asociados:** se pueden extraer subcadenas o valores parseados.

```
#[regex(r"[0-9]+", |lex| lex.slice().parse::<i64>().unwrap())]
Integer(i64),
```

- **Prioridad de patrones:** si dos patrones coinciden, se usa el de mayor prioridad.

- **Manejo de errores:** se puede definir un tipo de error personalizado.

```
#[logos(error = LexicalError)]
```

- **Depuración:** se puede habilitar la generación de gráficos de estados.

3.2 DESCRIPCIÓN DEL LENGUAJE L: MINIRUST

MiniRust es un subconjunto del lenguaje Rust que incluye las siguientes características:

3.2.1 Estructura del Lenguaje

- **Funciones:** definidas con la palabra clave `fn`, seguida del nombre, parámetros entre paréntesis y un bloque de código entre llaves.
- **Variables:** declaradas con `let`, opcionalmente con tipo y valor inicial.
- **Tipos de datos soportados:**
 - Enteros (`i32`, `i64`)
 - Booleanos (`bool`)
 - Flotantes (`f64`)
 - Operadores:
 - Aritméticos: `+`, `-`, `*`, `/`, `%`
 - Comparación: `==`, `!=`, `<`, `>`, `<=`, `>=`
 - Asignación: `=`
 - Estructuras de control:
 - `if / else`
 - `while`
 - `return`
 - Comentarios: de línea (`//`) y de bloque (`/* */`).
 - Literales: números enteros, flotantes, cadenas ("`...`"), booleanos (`true`, `false`).

3.2.2 Ejemplo de Programa en MiniRust

```
fn factorial(n: i64) -> i64 {  
    if n <= 1 {  
        return 1;  
    }  
}
```

```

    } else {
        return n * factorial(n - 1);
    }
}

fn main() {
    let result = factorial(5);
    // Esto es un comentario
    println!("Resultado: {}", result);
}

```

3.2.3 Tokens Definidos para MiniRust

Token	Descripción	Expresión Regular / Literal
Fn	Palabra clave fn	"fn"
Let	Palabra clave let	"let"
If	Palabra clave if	"if"
Else	Palabra clave else	"else"
While	Palabra clave while	"while"
Return	Palabra clave return	"return"
True / False	Booleanos	"true", "false"
Identifier	Identificadores	[a-zA-Z_][a-zA-Z0-9_]*
Integer	Números enteros	[0-9]+
Float	Números flotantes	[0-9]+\.[0-9]+
String	Cadenas	"([^\\"\\] \\.)*"

Operator	Operadores (+, -, *, /, etc.)	"+", "-", etc.
Assign	Asignación =	" = "
LParen / RParen	Paréntesis	" (", ") "
LBrace / RBrace	Llaves	" {", " } "
Semicolon	Punto y coma	" ; "
Comma	Coma	" , "
Comment	Comentarios (omitidos)	// .* o /*.**/

3.3 PROCESO DE CREACIÓN DEL LEXER CON LOGOS

3.3.1 Configuración del Proyecto

1. Crear un nuevo proyecto Rust:

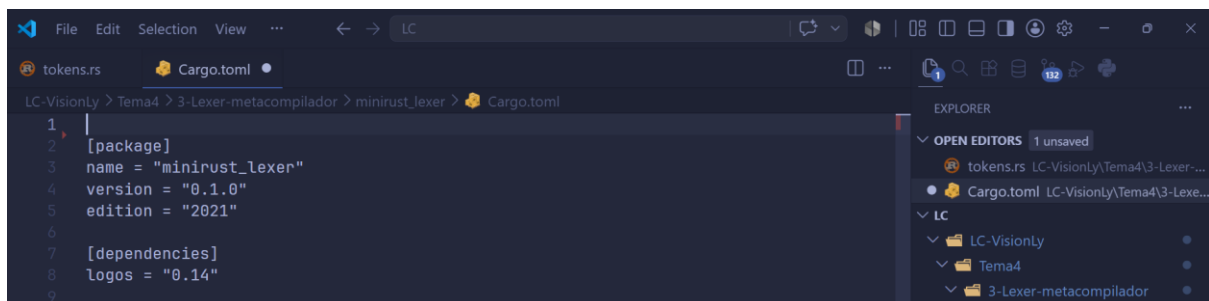


```

usuario@AnaS-PC MINGW64 /c/academico/LC/LC-VisionLy/Tema4/3-Lexer-metacompilador (main
)
$ cargo new minirust_lexer
cd minirust_lexer

```

2. Agregar la dependencia Logos en Cargo.toml:



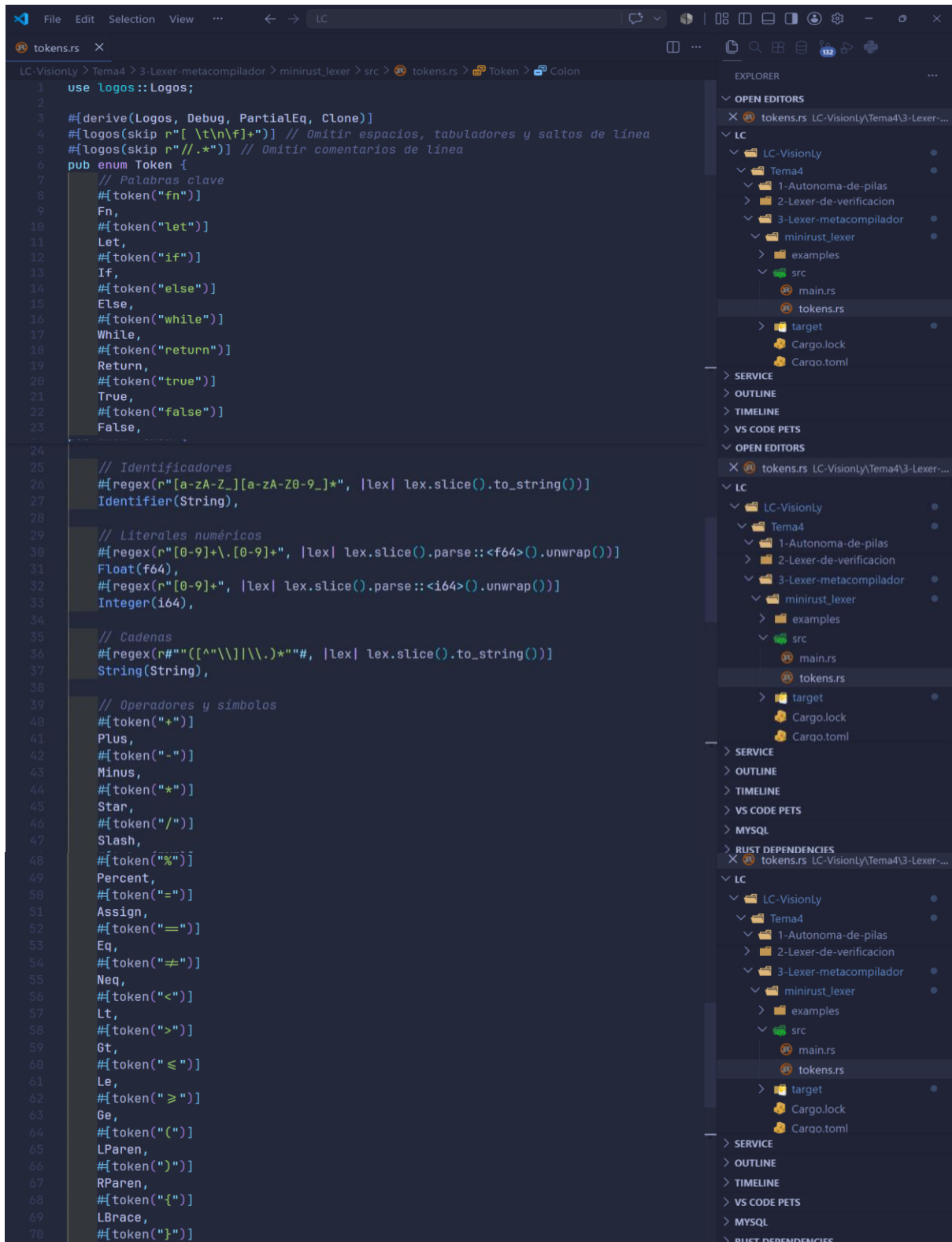
```

1 |
2 | [package]
3 | name = "minirust_lexer"
4 | version = "0.1.0"
5 | edition = "2021"
6 |
7 | [dependencies]
8 | logos = "0.14"
9 |

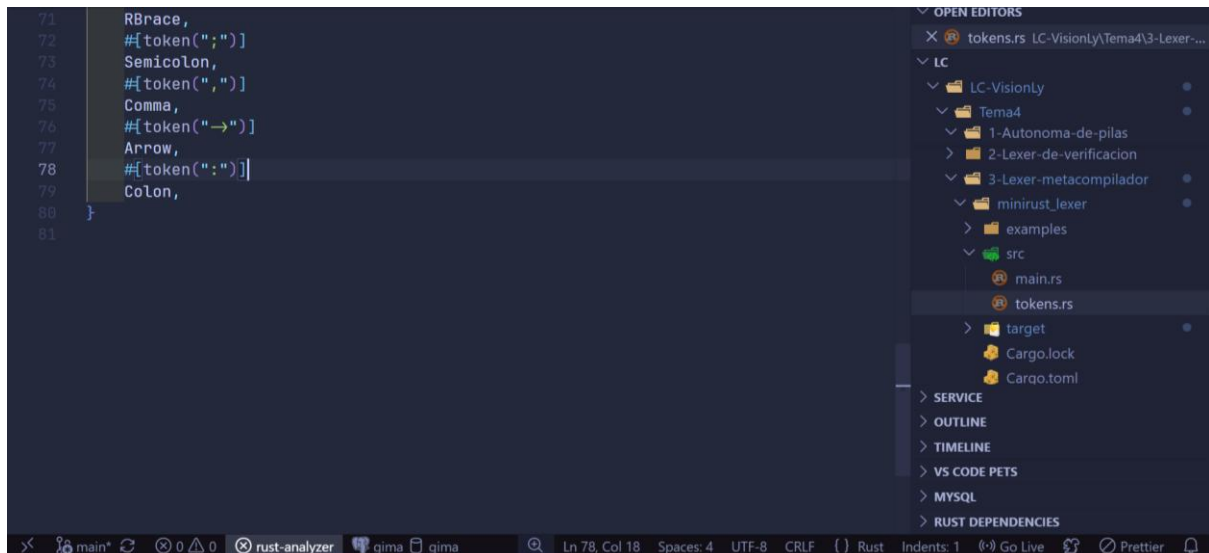
```

3.3.2 Definición de Tokens

En el archivo src/tokens.rs se define el enum Token:

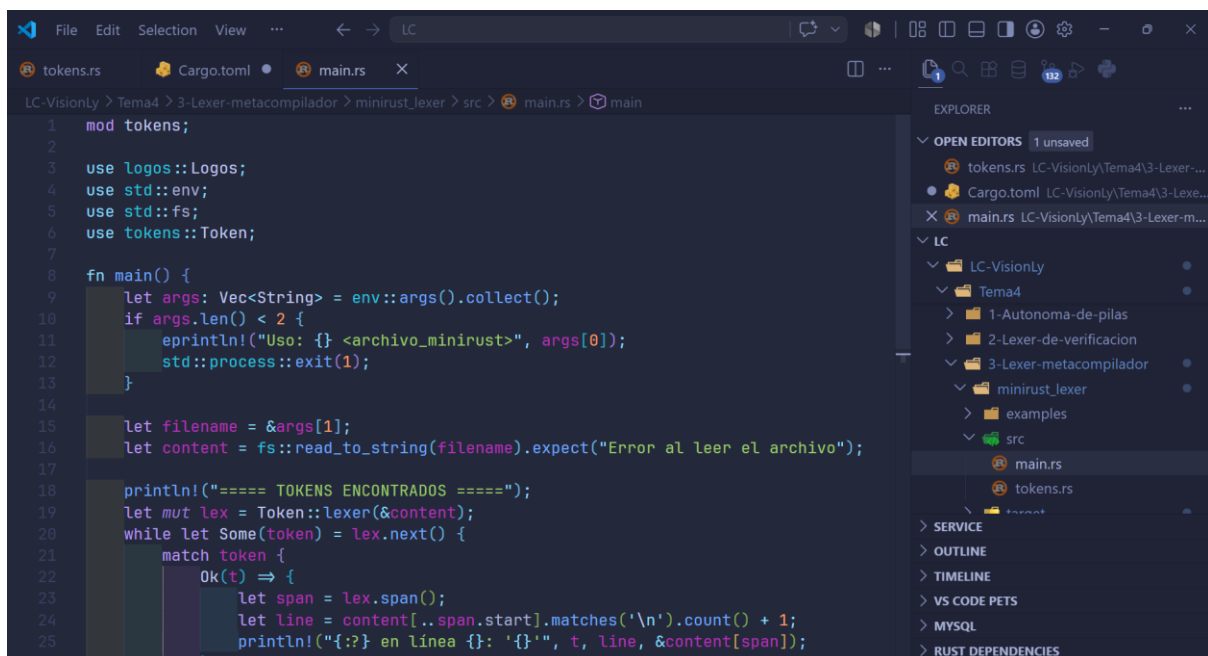


```
1 use logos::Logos;
2
3 #[derive(Logos, Debug, PartialEq, Clone)]
4 #[logos(skip r"[ \t\n\f]+" )] // Omitir espacios, tabuladores y saltos de línea
5 #[logos(skip r"//[.*)" ]] // Omitir comentarios de línea
6 pub enum Token {
7     // Palabras clave
8     #[token("fn")]
9     Fn,
10    #[token("let")]
11    Let,
12    #[token("if")]
13    If,
14    #[token("else")]
15    Else,
16    #[token("while")]
17    While,
18    #[token("return")]
19    Return,
20    #[token("true")]
21    True,
22    #[token("false")]
23    False,
24
25    // Identificadores
26    #[regex(r"[a-zA-Z_][a-zA-Z0-9_]*", |lex| lex.slice().to_string())]
27    Identifier(String),
28
29    // Literales numéricos
30    #[regex(r"[0-9]+\.[0-9]*", |lex| lex.slice().parse::<f64>().unwrap())]
31    Float(f64),
32    #[regex(r"[0-9]+", |lex| lex.slice().parse::<i64>().unwrap())]
33    Integer(i64),
34
35    // Cadenas
36    #[regex(r#"("[^"\\]|\\.)*"#, |lex| lex.slice().to_string())]
37    String(String),
38
39    // Operadores y símbolos
40    #[token("+")]
41    Plus,
42    #[token("-")]
43    Minus,
44    #[token("*")]
45    Star,
46    #[token("/")]
47    Slash,
48    #[token("%")]
49    Percent,
50    #[token("=")]
51    Assign,
52    #[token("==")]
53    Eq,
54    #[token("!=")]
55    Neq,
56    #[token("<")]
57    Lt,
58    #[token(">")]
59    Gt,
60    #[token("<=")]
61    Le,
62    #[token(">=")]
63    Ge,
64    #[token("(")]
65    LParen,
66    #[token(")")]
67    RParen,
68    #[token("{")]
69    LBrace,
70    #[token("}")]
```



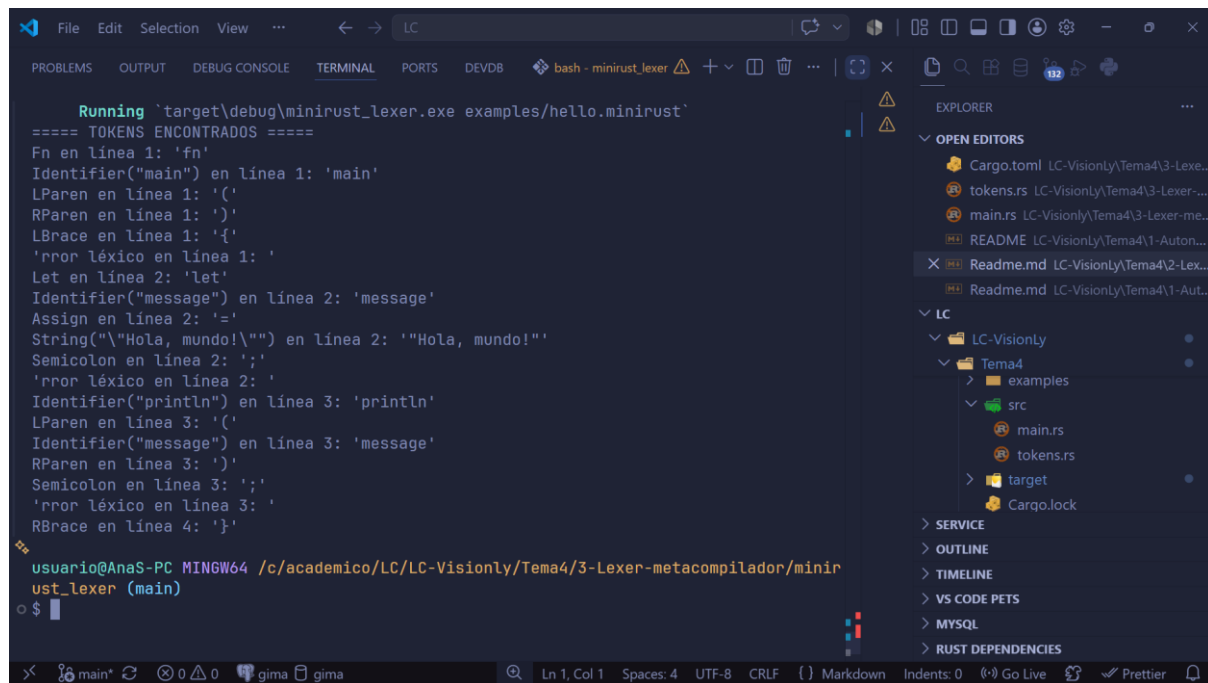
3.3.3 Implementación del Lexer

En *src/main.rs* se implementa la lógica para leer un archivo fuente y tokenizarlo:



EJEMPLO DE EJECUCIÓN

```
fn main() {  
    let message = "Hola, mundo!";  
    println(message);  
}
```



```
Running 'target\debug\minirust_lexer.exe examples\hello.minirust'  
==== TOKENS ENCONTRADOS ====  
Fn en línea 1: 'fn'  
Identifier("main") en línea 1: 'main'  
LParen en línea 1: '('  
RParen en línea 1: ')'  
LBrace en línea 1: '{'  
'rror léxico en línea 1: '  
Let en línea 2: 'let'  
Identifier("message") en línea 2: 'message'  
Assign en línea 2: '='  
String("\\"Hola, mundo!\\"") en línea 2: '"Hola, mundo!"'  
Semicolon en línea 2: ';'   
'rror léxico en línea 2: '  
Identifier("println") en línea 3: 'println'  
LParen en línea 3: '('  
Identifier("message") en línea 3: 'message'  
RParen en línea 3: ')'  
Semicolon en línea 3: ';'   
'rror léxico en línea 3: '  
RBrace en línea 4: '}'  
  
usuario@AnaS-PC MINGW64 /c/academico/LC/LC-VisionLy/Tema4/3-Lexer-metacompilador/minirust_lexer (main)  
$
```

4) LENGUAJES DE CIBERSEGURIDAD DONDE SE PUEDE USAR FLEX

La herramienta FLEX es un generador de analizadores léxicos que permite transformar especificaciones basadas en expresiones regulares en código ejecutable en C o C++, facilitando el reconocimiento de patrones complejos de manera eficiente. En el área de la seguridad informática, FLEX tiene aplicaciones críticas en el análisis de archivos de configuración, el procesamiento de registros de auditoría (logs) y la implementación de lenguajes de reglas para sistemas de detección de intrusos.

La aplicación de FLEX en seguridad informática permite construir herramientas de análisis de infraestructura que validan la integridad de configuraciones antes de su implementación en sistemas críticos. Al generar un Autómata Finito Determinista (AFD), FLEX garantiza un alto rendimiento en el procesamiento carácter por carácter, lo cual es esencial cuando se analizan grandes volúmenes de datos de tráfico de red o bitácoras de eventos en tiempo real.

Entre los lenguajes y contextos del área de seguridad donde se puede aplicar FLEX se encuentran:

- **Lenguaje de Configuración de Red (Debian interfaces)**

Un lenguaje relevante en seguridad es el utilizado en el archivo `/etc/network/interfaces` de sistemas Linux (como Debian), el cual define la arquitectura de red de un servidor, incluyendo segmentación, direccionamiento y parámetros de seguridad de red. Un error léxico o sintáctico en este lenguaje puede comprometer la conectividad o la seguridad perimetral del sistema.

Tokenización del Lenguaje de Interfaces

La tokenización consiste en convertir la corriente de caracteres del archivo en una secuencia de componentes léxicos o tokens dotados de significado. A continuación, se presenta la respectiva tokenización para este lenguaje basada en una especificación FLEX:

Expresión Regular (Patrón)	Token (Componente Léxico)	Descripción
auto	AUTO	Inicia la interfaz al arrancar el sistema.
iface	IFACE	Define una nueva interfaz de red.
address	ADDRESS	Palabra clave para asignar la IP.
netmask	NETMASK	Define la máscara de subred.
gateway	GATEWAY	Define la puerta de enlace predeterminada.
+\.+\.+\.+	IP	Formato de dirección IPv4.
[a-zA-Z0-9_-]+	INTERFACE_NAME	Nombre técnico de la interfaz (ej. eth0).
#. *	COMMENT	Ignora comentarios de documentación.
[\t\n]+	SKIP	Ignora espacios y saltos de línea.

- **Lenguaje de Especificación de Contenedores (Dockerfiles)**

Desde el punto de vista de seguridad, un lexer puede detectar el uso de usuarios no permitidos (como root) o la exposición de puertos sensibles.

Tokenización:

Instrucciones: FROM, RUN, CMD, EXPOSE, USER, COPY.

Argumentos: Cadenas de texto que representan rutas de archivos o nombres de imágenes.

Variables: Patrones que identifican \${VARIABLE}.

- **Análisis de Lenguajes con Enfoque en Seguridad (Rust)**

Aunque Rust es un lenguaje de propósito general, se debe destacar que representa el nuevo estándar de seguridad de memoria para la construcción del kernel Linux. Se puede aplicar FLEX para crear herramientas de análisis estático que busquen patrones de código inseguro dentro de subconjuntos del lenguaje Rust.

Elementos de Tokenización para un subconjunto de Rust:

- **Palabras Clave:** let, mut, fn, match, unsafe.
- **Tipos:** i32, u64, String, bool.
- **Operadores de Propiedad (Ownership):** &, & mut, *.

Utilidad de FLEX en Seguridad

La aplicación de FLEX permite construir reconocedores rápidos que procesan la entrada carácter por carácter. En seguridad, esto garantiza que el sistema pueda reaccionar en tiempo real ante patrones de ataque conocidos en el tráfico de red o identificar configuraciones riesgosas antes de que se apliquen al sistema, minimizando la superficie de ataque debida a errores morfológicos o sintácticos

CONCLUSIÓN

El dominio de los autómatas de pila y el análisis léxico es fundamental para el desarrollo de software estructurado, ya que permite procesar gramáticas complejas mediante una memoria auxiliar LIFO. El autómata de pila representa un avance cualitativo sobre los autómatas finitos al ser el reconocedor natural de los lenguajes libres de contexto, lo que facilita la validación de estructuras jerárquicas como el equilibrio de paréntesis y la gestión de llamadas a funciones. Asimismo, la implementación práctica de lexers para la verificación de archivos Docker mediante expresiones regulares demuestra cómo la teoría se traduce en herramientas de automatización capaces de detenerse ante errores léxicos inesperados. Por otro lado, el uso de metacompiladores como Logos para el subconjunto MiniRust evidencia una tendencia hacia la eficiencia y la seguridad de memoria, permitiendo generar analizadores rápidos mediante la definición de tokens y atributos específicos.

Finalmente, la aplicación de generadores como FLEX en ciberseguridad subraya que un análisis léxico riguroso es crítico para proteger la infraestructura, permitiendo validar la integridad de configuraciones de red y detectar patrones de código inseguro en tiempo real. De modo que, la integración de estos modelos teóricos con tecnologías modernas permite al futuro ingeniero construir sistemas que son, simultáneamente, jerárquicos, legibles y altamente seguros.

REFERENCIAS

Python Software Foundation. (s.f.). *re — Regular expression operations*. Recuperado de: <https://docs.python.org/3/library/re.html>

Docker Inc. (s.f.). *Dockerfile reference*. Recuperado de: <https://docs.docker.com/engine/reference/builder/>

IIEC. (s.f.). *Pydroid 3 - IDE for Python 3*. Aplicación móvil para Android. Recuperado de: <https://play.google.com/store/apps/details?id=ru.iiec.pydroid3>

RegExr: <https://regexr.com/>

Regex101: <https://regex101.com/>

Logos Handbook. (s.f.). Recuperado de <https://logos.maciej.codes/>

logos - Rust. (s.f.). Recuperado de <https://docs.rs/logos/latest/logos/>